شارك

SHARE-K

AI-Powered Open Source Collaboration Hub

Capstone Project — Full Technical Documentation

Track	Post Graduate Full Stack Development
Institution	ICT Training Institution (ITI)
Version	1.0 — January 2026
Target Tier	**GOLD — Industry-Leading (90–100%)**

Connecting Developers. Validating Skills. Powering Open Source.

Table of Contents

1. Executive Summary

Share-k is an AI-powered open-source collaboration hub that solves a critical bottleneck in the developer ecosystem: the mismatch between open-source project owners who need contributors and developers who want to build real-world experience. Traditional platforms like GitHub offer no structured way to match, validate, or onboard contributors intelligently.

Share-k addresses this with a full-stack, AI-first platform that uses LLM-driven skill profiling, RAG-powered contributor matching, and a multi-agent orchestration system — all working behind a clean, intuitive interface.

Problem Solved	Unstructured contributor discovery in open-source projects
Target Users	Open-source project owners & developers seeking real experience
Core AI Value	Automated skill extraction, intelligent matching, and gap guidance
Market Category	Developer Tools & Productivity / Knowledge Management
MVP Timeline	4 weeks, 5-member team
Target Eval Tier	GOLD (90–100%) — Industry-Leading

2. Problem Statement & Solution

2.1 The Problem

Open-source development powers the modern tech industry, yet it suffers from severe collaboration inefficiencies:

- Project owners post contribution tasks but receive applications from developers who lack the required skills, wasting hours reviewing unsuitable candidates.
- Developers struggle to discover real projects matching their actual skill level, stifling career growth and portfolio development.
- There is no trusted, verified reputation system for contributor quality in the open-source space outside of raw GitHub metrics.
- Developers who are rejected receive no actionable guidance on how to improve or become eligible.

2.2 The Solution — Share-k

Share-k eliminates these pain points with four AI-driven pillars:

Pillar	Capability	Technology
Smart Skill Profiling	Automatically analyze GitHub repos and generate verified skill profiles	LLM + GitHub API + RAG
Intelligent Matching	Match contributors to tasks based on actual technical fit	Agentic AI + Vector Search
Validation Gating	Block unqualified applications before they reach project owners	AI Skill Validation Agent
Growth Guidance	Provide personalized learning paths to close skill gaps	RAG + LLM Generation

3. User Roles & Permissions

The Share-k MVP is structured around three distinct user roles, each with a clearly defined scope of interaction with the platform and its AI systems.

3.1 Project Owner

Developers or teams who publish open-source projects and define contribution tasks.

- Connect GitHub account; system auto-fetches repository data
- Publish and manage projects on the platform
- Create contribution orders (tasks) with title, description, required technologies, difficulty, deadline, and optional reward
- Review incoming contributor applications pre-filtered by AI validation
- Accept, reject, and provide ratings and feedback on delivered contributions
- Access premium subscription tiers for expanded order limits and AI-powered top-N contributor matching

3.2 Contributor

Developers who seek to gain experience, build a verified reputation, and contribute to real projects.

- Connect GitHub account; AI generates an initial skill profile from repository analysis
- Browse and filter available projects and contribution orders
- Apply for tasks; application is gated by an AI skill validation check
- Submit work via GitHub Pull Request link upon task completion
- Build a public reputation profile based on completed work, ratings, and success rate
- Premium Gold tier receives AI-generated skill gap feedback on rejected applications

3.3 Admin / Support

Platform administrators who ensure trust, quality, and safety.

- Review and approve AI-generated skill profiles before account activation
- Validate the accuracy of AI skill assessments across the user base
- Review reported issues and moderate platform content
- Prevent fraud and misuse — a critical trust layer for the reputation economy

4. Core Features (MVP)

4.1 User Registration & AI Skill Profile Generation

When a user registers and connects their GitHub account, an automated pipeline activates:

- GitHub API fetches: repositories, programming languages, contribution history, technologies used in each project
- The AI Skill Profiling Agent analyzes this data and generates a structured skill profile with proficiency levels (e.g., Python – Advanced, React – Intermediate, Docker – Beginner)
- The generated profile is held in a pending state and reviewed by an Admin before the account becomes fully active, ensuring platform trust

4.2 Project Publishing

Project owners publish projects via GitHub URL. The system automatically populates metadata using the GitHub API:

- Project title, description, programming languages, tags/technologies, repository statistics
- Owner reviews and edits auto-populated data before confirming publication
- Published projects are indexed in the vector database for semantic discovery

4.3 Project Discovery

Contributors browse a discovery feed of all published projects with multi-dimensional filtering:

- Filter by technology stack, project category (Web Dev, Mobile, AI/ML, DevOps, Tools & Utilities), and difficulty level
- Semantic search powered by vector embeddings surfaces projects beyond keyword matching

4.4 Contribution Requests (Orders)

Project owners create structured task orders that define the scope of needed contributions:

- Fields: title, description, required technologies, difficulty level, deadline, optional monetary reward
- Orders are indexed for AI-based matching and appear in the contributor task feed

4.5 AI-Gated Contributor Applications

This is Share-k's core AI feature and primary differentiator. When a contributor applies to a task:

- The AI Skill Validation Agent compares the contributor's verified skill profile against the task's requirements
- Eligible contributors: application is forwarded to the project owner for final review and acceptance
- Ineligible contributors: the system blocks the application and informs the user they do not currently meet the requirements
- Gold-tier contributors: additionally receive a detailed AI-generated guidance report identifying missing skills, recommended technologies, and suggested practice projects

4.6 Contribution Delivery & Review

- Accepted contributors submit a GitHub Pull Request link as proof of completed work
- Project owner reviews and approves or rejects the submission
- Upon approval, the owner rates the contributor and leaves feedback

4.7 Reputation System

Each contributor builds a transparent, verifiable public profile:

Metric	Description
Overall Rating	Aggregate score from project owner reviews (e.g., 4.8/5.0)
Tasks Completed	Total number of approved contributions
Success Rate	Percentage of accepted vs. submitted contributions
Top Skills	Most frequently used and validated skills

4.8 Premium Subscription Tiers

Owner Plans

Plan	Monthly Orders	AI Matching	Visibility	Commission
Bronze	10 orders/month	—	Standard	Standard
Silver	20 orders/month	Top 5 matches	Priority listing	Standard
Gold	30 orders/month	Top 10 matches	Priority + auto-notify best matches	0% on payments

Contributor Plans

Plan	Daily Applications	Task Notifications	Commission	AI Gap Feedback

Bronze	2 per day	Basic	Standard	—
Silver	3 per day	Skill-matched	Reduced	—
Gold	4 per day	AI-recommended	0%	Full guidance report

5. AI Architecture — Mapping to Checklist Requirements

This section maps every AI feature in Share-k directly to the mandatory and recommended requirements in the ICT Capstone checklist, demonstrating Gold-tier compliance.

5.1 LLM Integration & API Layer

Share-k integrates LLMs at every stage of the user journey — from onboarding to task matching to skill guidance.

Primary LLM Provider

Primary Provider	OpenAI GPT-4o — chosen for function calling reliability, structured JSON output, and strong code-understanding capability
Fallback Provider	Anthropic Claude 3 Haiku — cost-efficient fallback for lower-stakes tasks (notifications, summaries)
Embedding Model	OpenAI text-embedding-3-small — for all vector operations
API Key Management	All keys stored in environment variables; rotated via AWS Secrets Manager

Core LLM Capabilities Used in Share-k

- Text Generation: skill profile summaries, task descriptions, feedback narratives
- Structured Output (JSON mode): skill extraction returns structured JSON {skill: string, level: string, confidence: float}
- System Prompts: each agent has a carefully designed system prompt with role definition, constraints, and output format specification
- Context Management: multi-turn conversation history maintained per session for contributor guidance chats
- Token Management: token budgets enforced per request type; cached embeddings reduce redundant API calls

Advanced API Features Implemented

- Function Calling (Tool Use): the Skill Validation Agent uses function calling to query the contributor profile database and task requirement store
- Streaming Responses: skill gap feedback and project descriptions stream token-by-token in the UI for a responsive experience

5.2 RAG (Retrieval-Augmented Generation) Implementation

RAG is the backbone of Share-k's intelligence. It powers three distinct use cases: skill profile generation from GitHub data, contributor-to-task matching, and skill gap guidance generation.

Where RAG is Used in Share-k

RAG Use Case	Data Source	Purpose
Skill Profile RAG	GitHub README files, code files, commit messages, repo descriptions	Extract skills from unstructured developer activity data
Contributor Matching RAG	Indexed contributor skill profiles + task requirement documents	Retrieve top-N best-fit contributors for a given task (Owner Gold plan)
Skill Gap Guidance RAG	Curated learning resource index + technology documentation summaries	Generate personalized learning paths for rejected Gold contributors
Project Discovery RAG	Published project metadata and descriptions	Surface semantically relevant projects to contributors browsing the platform

Document Processing Pipeline

- Multi-format Support: GitHub README (Markdown), code files (.py, .js, .ts, .go), DOCX/PDF for user-uploaded portfolios, plain text
- Document Loader: automated ingestion triggered on GitHub OAuth connection; re-runs on profile refresh
- Text Extraction: OCR support via AWS Textract for scanned resumes/certificates (optional upload)
- Metadata Extraction: repository name, stars, last commit date, primary language, contributor count

Chunking Strategy

- Intelligent Chunking: semantic chunking applied to README/documentation content; file-level chunking for code analysis
- Chunk Size: 512–768 tokens per chunk, empirically tested for skill extraction tasks
- Overlap: 15% overlap between adjacent chunks to prevent skill context from being split across boundaries
- Context Preservation: headings and section titles preserved as chunk metadata for attribution

Vector Database & Embeddings

Vector Store	Pinecone (managed) — chosen for serverless scaling and metadata

	filtering support
Embedding Model	OpenAI text-embedding-3-small (1536 dimensions)
Indexing Strategy	HNSW (Hierarchical Navigable Small World) — optimal for approximate nearest-neighbor search at scale
Namespaces	Separate namespaces for: contributor_profiles, task_requirements, project_metadata, learning_resources
Metadata Filtering	Filter by skill_category, difficulty_level, technology_tag, last_updated

Retrieval Strategy (Hybrid — 2 approaches)

- Semantic Search: dense vector similarity search for contributor-task matching and project discovery
- Hybrid Retrieval: combined semantic + BM25 keyword search with Reciprocal Rank Fusion (RRF) for skill gap resource retrieval, ensuring both concept-level and exact-term matches are captured
- Reranking: a cross-encoder reranker (ms-marco-MiniLM) re-scores the top-20 retrieved candidates before presenting top-5 to the LLM — critical for matching quality

RAG Quality & Evaluation

- Context Precision: measured — relevance of retrieved skill chunks to the queried task
- Context Recall: measured — completeness of skill coverage from developer's GitHub history
- Faithfulness: all AI-generated skill profiles cite the specific repository/file that evidences each skill
- Answer Relevancy: skill gap guidance responses measured against task requirement specificity
- Citation/Source Attribution: every generated skill profile shows the source repository for each skill

Advanced RAG Feature — Query Transformation

When a contributor applies to a task, the system applies Query Transformation: the task description is reformulated into multiple targeted sub-queries (e.g., 'Node.js JWT authentication intermediate' decomposes into queries for Node.js proficiency evidence, JWT usage examples, and authentication pattern familiarity) to maximize retrieval recall across the contributor's code history.

5.3 Agentic AI Implementation

Share-k uses a multi-agent system built on LangChain/LangGraph. Each agent has a specific role, a defined set of tools, and clear input/output contracts. The agents work in a sequential pipeline architecture.

Agent 1: GitHub Ingestion & Skill Profiling Agent

Role	Analyzes a new user's GitHub account and generates their initial skill profile
Trigger	User connects GitHub account during registration
Tools	github_fetch_repos, github_fetch_readme, github_fetch_languages, vector_upsert_profile, structured_skill_extractor
Output	Structured JSON skill profile: [{skill, level, confidence, source_repo}]
Memory	Short-term: session context; Long-term: profile stored in PostgreSQL + Pinecone
Architecture	ReAct (Reason + Act) — iterates across repositories, reasoning about evidence

Agent 2: Skill Validation Agent

Role	Validates whether a contributor's skill profile satisfies a task's requirements before forwarding the application
Trigger	Contributor clicks 'Apply' on a task
Tools	fetch_contributor_profile, fetch_task_requirements, semantic_skill_comparator, eligibility_decision
Output	Binary decision: ELIGIBLE (forward to owner) or INELIGIBLE (block + notify) with confidence score and justification
Reasoning	Chain-of-thought: lists required skills, checks each against profile, computes overall eligibility
Error Handling	On tool failure, defaults to human review queue rather than auto-approving

Agent 3: Skill Gap Guidance Agent (Gold Tier)

Role	Generates a personalized learning roadmap for a contributor who was rejected by the Skill Validation Agent
Trigger	INELIGIBLE decision + contributor has Gold subscription
Tools	fetch_missing_skills_report, rag_retrieve_learning_resources, generate_roadmap, fetch_practice_projects
Output	Structured guidance: {missing_skills: [], recommended_resources: [], practice_projects: [], estimated_weeks: int}
RAG Usage	Retrieves from the learning_resources namespace in Pinecone — curated documentation links, tutorials, and open projects

Agent 4: Contributor Matching Agent (Owner Gold/Silver Tier)

Role	Proactively identifies and ranks the top-N best-fit contributors for a newly published task order
Trigger	Project owner with Silver/Gold plan publishes a new contribution order
Tools	fetch_task_requirements, vector_search_contributors, rank_candidates, notify_top_contributors
Output	Ranked list of contributor profiles with match score and skill alignment summary
Multi-agent Coordination	Receives task embedding from the Ingestion Agent; passes ranked list to the Notification Service

Multi-Agent Architecture: Sequential Pipeline

The four agents operate as a sequential pipeline with a Supervisor Agent (LangGraph StateGraph) orchestrating transitions:

Stage	Agent	Input	Output
1 — Onboarding	GitHub Ingestion & Skill Profiling Agent	GitHub OAuth token	Verified skill profile (pending admin approval)
2 — Application	Skill Validation Agent	Contributor profile + Task requirements	ELIGIBLE / INELIGIBLE + justification
3a — If INELIGIBLE + Gold	Skill Gap Guidance Agent	Missing skills report	Personalized learning roadmap
3b — New Task Published	Contributor Matching Agent	Task requirements embedding	Top-N ranked contributor matches

Agent Memory & Context

- Short-term Memory: LangGraph session state — conversation and decision history within a single workflow run
- Long-term Memory: PostgreSQL stores all skill profiles, application decisions, and feedback history; retrievable across sessions
- Episodic Memory: past application decisions stored and surfaced to the Skill Validation Agent to detect repeated borderline cases for admin escalation
- Context Window Management: for long contributor histories, a summarization step condenses older activity before passing to the LLM context window

Advanced Agent Features

- Human-in-the-Loop: Admin review step is mandatory before skill profiles activate; critical matching decisions surface a confidence score — low-confidence cases are flagged for human review
- Agent Self-Reflection: the Skill Validation Agent includes a self-critique step where it re-evaluates its eligibility decision against the task requirements before finalizing, reducing false negatives
- Dynamic Replanning: if the GitHub Ingestion Agent encounters rate limits or unavailable repositories, it replans the ingestion order and processes available data first, then schedules retry for failed repos

6. Multimodal AI Capabilities

Share-k implements two multimodal capabilities, meeting the minimum requirement of 2.

6.1 Vision / Image Understanding — GitHub Profile & Repository Analysis

- Technology: GPT-4o Vision
- Use Case: when a contributor uploads a portfolio PDF, certificate image, or a screenshot of a project UI, the Vision model analyzes the image content and extracts additional skill evidence (e.g., identifying a React UI screenshot as evidence of frontend competency)
- Supported Formats: JPG, PNG, WebP, PDF (page-by-page rendering)
- Integration: output of Vision analysis is merged into the contributor's RAG skill profile alongside GitHub data

6.2 Speech-to-Text — Accessibility for Contribution Task Creation

- Technology: OpenAI Whisper API
- Use Case: project owners can dictate contribution task descriptions via voice input, which Whisper transcribes into the task creation form — improving accessibility and reducing friction for non-native writers
- Language Support: English and Arabic (bilingual platform requirement)
- Mode: batch transcription triggered on audio file upload; real-time STT available as enhanced feature

7. LLM Orchestration & Framework

7.1 Primary Framework: LangGraph

LangGraph was selected as the primary orchestration framework for its stateful multi-agent workflow support, native LangChain integration, and ability to express complex conditional branching (e.g., eligibility decision routing).

7.2 Framework Architecture

- Modular Architecture: retrieval, generation, and orchestration layers are fully separated — swapping the vector store or LLM provider requires zero changes to agent logic

- Chain Management: each agent's LangChain chain is encapsulated in its own module with clear input/output schemas (Pydantic models)
- Error Boundaries: every node in the LangGraph StateGraph has a try/except wrapper; failures route to a dedicated error-handling node
- Logging Integration: all LangGraph trace events stream to Langfuse for full observability
- Testing Suite: pytest with LangChain mock LLM for deterministic unit tests on all agent chains

7.3 Prompt Management

- Centralized Prompts: all system prompts stored in a dedicated prompts/ directory and loaded at runtime — never hardcoded
- Version Control: prompt versions tracked in Git with semantic versioning (prompts/skill_validation/v2.1.txt)
- Prompt Templates: Jinja2 parameterized templates with variables for task requirements, contributor skills, and platform context
- Few-shot Examples: skill extraction and validation prompts include 3–5 curated examples covering edge cases (e.g., contributor with tangential skills)

8. Security, Safety & Compliance

8.1 OWASP LLM Top 10 2025 Coverage

#	Threat	Share-k Mitigation
1	Prompt Injection	Input sanitization on all user-submitted text; system prompt protection via Llama Guard screening; indirect injection defense in GitHub README ingestion pipeline
2	Sensitive Info Disclosure	PII detection (presidio library) on all LLM outputs before display; skill profiles strip personal identifiers; audit log of all data access
3	Supply Chain	Dependency scanning (Dependabot); all model sources verified (OpenAI official API only); version-pinned requirements.txt
4	Excessive Agency	Agent actions rate-limited; no agent can modify contributor profiles without human-approved trigger; all destructive actions require confirmation
5	Insecure Output Handling	All LLM-generated content sanitized before rendering (DOMPurify for web); no code execution from generated content; XSS prevention via CSP headers

8.2 Guardrails & Content Moderation

- Input Guardrails: OpenAI Moderation API screens all user-submitted task descriptions and profile content
- Output Guardrails: LLM responses checked for hallucinated skill levels before display; confidence threshold enforced
- PII Detection: presidio library identifies and redacts names, emails, and phone numbers from ingested GitHub data

8.3 Rate Limiting & Abuse Prevention

- Per-user API rate limits enforced at the application layer (FastAPI middleware)
- Global rate limits on LLM API calls with exponential backoff and circuit breaker
- Token budgets: max 2,000 tokens per skill extraction request; max 4,000 for gap guidance
- Premium tier application limits (2/3/4 per day) enforced at database level, not just UI

8.4 Data Privacy & Compliance

- All data encrypted in transit (TLS 1.3) and at rest (AES-256 on PostgreSQL and S3)
- GitHub OAuth scopes limited to read-only public data — no write access ever requested
- Users can request full data deletion; profile data purged from PostgreSQL and Pinecone namespaces
- GDPR/CCPA readiness: data retention policy defined (24 months active, 6 months post-deletion grace period)

9. Monitoring, Observability & Evaluation

9.1 LLM Observability Platform: Langfuse

Langfuse provides full trace visibility across all agent actions, LLM calls, retrieval steps, and tool invocations. Every application in Share-k generates a complete trace from user action to final response.

9.2 Quality Evaluation Metrics (5 implemented — exceeds minimum of 3)

Metric	What It Measures	Target	Where Applied
Context Precision	Relevance of GitHub code chunks retrieved for skill extraction	> 85%	Skill Profiling RAG
Context Recall	How completely a contributor's skill set is captured from their repos	> 80%	Skill Profiling RAG
Faithfulness	Skill level claims grounded in retrieved code evidence	> 90%	All RAG outputs
Answer Relevancy	Gap guidance addresses the specific missing skills identified	> 85%	Skill Gap Agent
Task Success Rate	% of contributor applications correctly classified (ELIGIBLE/INELIGIBLE)	> 90%	Skill Validation Agent

9.3 LLM-as-a-Judge

- Automated Scoring: GPT-4o judges the quality of skill gap guidance reports on a 1–5 rubric covering specificity, actionability, and relevance
- Pairwise Comparison: used during prompt iteration to compare v1 vs. v2 guidance quality before deployment

9.4 Human Evaluation

- Thumbs Up/Down: inline on every AI-generated skill profile and guidance report
- Star Ratings: contributors rate the accuracy of their generated skill profiles post-admin-approval
- Human Labeling Queue: Admins reviewing skill profiles provide ground-truth labels that feed back into evaluation datasets

9.5 Performance & Cost Monitoring

- Latency Tracking: P95 target < 3 seconds for skill validation; < 8 seconds for full guidance report generation
- Token Usage Dashboard: real-time cost attribution per agent, per user tier, per model
- Cost Alerts: Slack notifications when daily LLM spend exceeds 80% of budget threshold
- Error Rate Tracking: failures categorized by type (GitHub API, OpenAI timeout, Pinecone unavailability)

10. User Experience & Interface

10.1 Conversational Interface

- Clean chat UI for the Skill Gap Guidance feature — contributors can ask follow-up questions about their learning path
- Markdown rendering for all AI-generated content (skill profiles, gap reports, task descriptions)
- Streaming responses: guidance reports stream token-by-token for immediate feedback
- Message history: full conversation with the guidance agent is persisted per session

10.2 File Upload & Management

- Drag-and-drop upload for portfolio PDFs and certificates (processed by Vision AI and OCR)
- Multi-file support with upload progress indicators
- File management panel: view, re-process, or delete uploaded portfolio documents

10.3 User Feedback Mechanisms

- Inline thumbs up/down on every AI-generated skill profile card
- 'Report Inaccuracy' flag for contributors to contest AI-assigned skill levels
- 'Request Review' for edge cases where contributors believe they meet task requirements

10.4 Advanced UI Features (3 implemented)

- Citation Bubbles: each skill in a generated profile shows a clickable bubble linking to the source repository that evidenced the skill
- Confidence Scores: skill levels show a confidence percentage derived from the amount and quality of code evidence retrieved
- Export Chat: contributors can export their skill gap guidance session as a PDF learning plan

10.5 Responsive Design & Accessibility

- Mobile-first responsive design; tablet and desktop layouts with adaptive navigation
- WCAG 2.1 Level AA: semantic HTML, ARIA labels, keyboard navigation, color contrast compliance, screen reader support
- RTL support for Arabic language users (both LTR/RTL layout modes available via language toggle)

11. Cost Optimization & Scalability

11.1 Cost Management

- Embedding Caching: once a contributor's GitHub repositories are embedded, the vectors are cached in Pinecone — re-runs only process new/changed repos
- Query Caching: identical skill validation requests (same contributor + same task) return cached decision within TTL window
- Model Tiering: GPT-4o used only for Skill Validation and Gap Guidance; GPT-3.5-Turbo handles simpler summarization and notification tasks
- Batch Processing: when a Gold owner publishes a task, contributor matching runs as a background batch job rather than real-time
- Cost Dashboard: admin panel shows live spend breakdown by agent, model, and user tier

11.2 Scalability Architecture

- Stateless FastAPI backend — horizontal scaling via container replication
- Celery + Redis queue for background agent tasks (skill profiling, batch matching) — decoupled from synchronous API responses
- Pinecone serverless scales automatically with query volume
- Connection pooling (pgBouncer) for PostgreSQL under concurrent load
- Circuit breakers on all external API calls (GitHub, OpenAI, Pinecone) via tenacity library

12. Testing & Quality Assurance

12.1 Unit Testing

- Backend: pytest with 60%+ coverage target; all agent chain functions individually tested with mock LLM responses
- Frontend: Vitest + React Testing Library for component tests
- LLM Mocking: LangChain FakeLLM and recorded responses used for deterministic agent tests

12.2 Integration Testing

- API endpoint tests with realistic payloads covering all user flows

- Vector DB integration: embedding and retrieval flows tested against a test Pinecone namespace
- GitHub API integration: mocked with responses covering rate limits, empty repos, and private-repo edge cases

12.3 LLM-Specific Testing

- Evaluation Dataset: 100+ test cases with ground-truth eligibility decisions (ELIGIBLE/INELIGIBLE) for the Skill Validation Agent
- Hallucination Testing: prompts designed to produce false skill claims tested with deliberately mismatched profiles
- Adversarial Testing: prompt injection attempts in task descriptions; jailbreak attempts in contributor bio fields
- Multi-language Testing: English and Arabic task descriptions and skill profiles tested end-to-end

12.4 E2E Testing

- Playwright test suite covering: contributor registration → skill profile generation → task discovery → application → eligibility decision
- Owner flow: project publishing → order creation → application review → contribution approval

13. Deployment & DevOps

13.1 Stack Overview

Backend	Python (FastAPI) — async, OpenAPI auto-generated docs
Frontend	React (Next.js) — SSR for SEO, Tailwind CSS, RTL support
AI Framework	LangChain + LangGraph (Python)
Vector Database	Pinecone (serverless)
Primary Database	PostgreSQL (AWS RDS)
Cache / Queue	Redis (ElastiCache) + Celery workers
Cloud Provider	AWS (ECS Fargate for backend, Amplify for frontend, S3 for files)
Observability	Langfuse (LLM traces) + CloudWatch (infrastructure)
Error Tracking	Sentry

13.2 CI/CD Pipeline (GitHub Actions)

- On every push: run pytest (backend) + Vitest (frontend); lint with Ruff and ESLint
- On merge to develop: build Docker image, push to ECR, deploy to staging environment
- On merge to main: deploy to production with automatic rollback on health check failure
- Branch strategy: main (production), develop (integration), feature/* (individual work)

13.3 Containerization

- Multi-stage Dockerfile: build stage (dependencies) → runtime stage (lean image, ~180MB)
- Docker Compose for local development: FastAPI + PostgreSQL + Redis + Langfuse
- Environment variables externalized; AWS Secrets Manager for production API keys

14. Innovation & Differentiation

14.1 Unique Value Proposition

Share-k is the only open-source collaboration platform that uses AI to validate contributor eligibility before applications reach project owners. This eliminates the #1 pain point for maintainers — wasted time reviewing unqualified candidates — while simultaneously creating a structured growth pathway for developers.

Dimension	Share-k	GitHub Issues / Existing Platforms
Contributor Matching	AI-powered skill validation + semantic matching	Manual review of all applicants
Skill Verification	AI-extracted from actual code, not self-reported	Self-declared skills only
Rejection Experience	Gold tier: AI learning roadmap generated instantly	No guidance provided
Reputation	Multi-metric, contribution-verified public profile	Star counts only

14.2 Advanced AI Technique: Constitutional AI-Inspired Self-Critique

The Skill Validation Agent implements a self-reflection loop inspired by Constitutional AI: after generating an initial eligibility decision, the agent is prompted to critique its own reasoning against a set of fairness principles (e.g., 'Did I penalize for skill gaps in technologies that are easy to learn on-the-job?'). This reduces false negatives and creates a more equitable matching system.

14.3 Arabic Language Support

- Full RTL UI with proper Arabic text rendering in all components
- Arabic task descriptions and skill profiles tested with GPT-4o's native Arabic language support
- Arabic voice input via Whisper for task creation (supports MSA)
- Bilingual platform toggle between English and Arabic throughout the interface

14.4 Model Context Protocol (MCP) — Stretch Goal

Share-k plans to expose its skill profiling capability as an MCP server, enabling IDE integrations (VS Code extension) to query a developer's Share-k skill profile directly from their development environment — opening a path to AI-assisted project discovery at the point of code authorship.

15. Documentation & Knowledge Transfer

15.1 Technical Documentation

- Architecture Diagram: full system component diagram with data flow annotations (included in repository /docs/architecture.png)
- API Documentation: auto-generated OpenAPI/Swagger spec via FastAPI; accessible at /api/docs in all environments
- Database Schema: ER diagram documenting all tables: users, skills, projects, orders, applications, reputation_events
- Deployment Guide: step-by-step instructions for local setup and cloud deployment

15.2 AI-Specific Documentation

- Prompt Library: all system prompts versioned and documented in /prompts/ with rationale for design choices
- Model Selection Rationale: documented comparison of GPT-4o vs. Claude 3 vs. Gemini for skill extraction task
- Evaluation Results: benchmark metrics from the 100+ test evaluation dataset for the Skill Validation Agent
- Known Limitations: skill extraction accuracy degrades for repositories with minimal README documentation; noted with mitigation guidance

15.3 User Documentation

- User Guide: illustrated walkthrough for contributor registration, project discovery, and application flow
- Owner Guide: how to publish projects, create orders, and interpret AI matching results
- FAQ: covering common questions about AI skill profile generation and how to contest results

16. Gold Tier Compliance Summary

The following table cross-references Share-k's implementation against the Gold Tier requirements of the ICT Capstone Evaluation Framework.

Requirement Category	Gold Tier Requirement	Share-k Implementation	Status
LLM Integration	Advanced API features (streaming, function calling)	Function calling in Skill Validation + Streaming in Gap Guidance	✓Complete
RAG	Advanced RAG (hybrid retrieval + reranking)	Semantic + BM25 hybrid with cross-encoder reranker	✓Complete
RAG Advanced	Advanced RAG feature (query transformation, multi-query, etc.)	Query Transformation on task applications	✓Complete
Agentic AI	Multi-agent system (3+ agents)	4 specialized agents in LangGraph sequential pipeline	✓Complete
Agentic AI	Advanced agent features (self-reflection, replanning)	Self-reflection in Validation Agent; Dynamic replanning in Ingestion Agent	✓Complete
Agentic AI	Human-in-the-loop	Admin approval gate on all skill profiles	✓Complete
Multimodal	2+ multimodal capabilities	Vision (GPT-4o) + Speech-to-Text (Whisper)	✓Complete
Orchestration	Production-ready framework	LangGraph with LangChain; modular, tested, logged	✓Complete
Security	OWASP LLM Top 10 coverage	All 5 primary threats addressed with specific mitigations	✓Complete
Monitoring	LLM observability platform + 3+ quality metrics	Langfuse + 5 quality metrics + LLM-as-a-Judge	✓Complete
Evaluation	LLM-as-a-Judge + human eval loop	GPT-4o judge + admin label queue feeding eval dataset	✓Complete
UX	Streaming, citations, feedback, advanced features	All implemented; citation bubbles + confidence scores	✓Complete
Innovation	Unique technique or emerging technology	Constitutional AI self-critique loop + MCP server roadmap	✓Complete
Arabic	Arabic language support	RTL UI + Arabic voice input + bilingual toggle	✓Complete
DevOps	CI/CD + cloud deployment + containerization	GitHub Actions + AWS ECS + multi-stage Docker	✓Complete
Testing	E2E + LLM evaluation dataset (50+ cases)	Playwright E2E + 100-case evaluation dataset	✓Complete

18. Success Metrics & KPIs

Category	Metric	Target
Technical	P95 API response time	< 3 seconds
Technical	Platform uptime during evaluation	≥ 99%
Technical	Test code coverage	> 60%
Technical	Critical security vulnerabilities	0
AI Quality	Skill Validation accuracy (vs. ground truth)	> 90%
AI Quality	RAG faithfulness score	> 90%
AI Quality	Gap guidance answer relevancy	> 85%
AI Quality	User satisfaction (thumbs up on profiles)	> 70%
Cost	Average cost per skill validation request	< $0.05
Business	Scalability	Handles 100+ concurrent users

19. Closing Notes

Share-k is designed from the ground up as a production-ready, market-viable platform — not a demo. Every AI feature directly solves a documented user pain point, and every architectural decision is justified by the constraints of a real deployment environment.

The platform's AI architecture is not additive window dressing: LLM-driven skill profiling, RAG-powered matching, and the multi-agent validation pipeline are the product. Without them, Share-k is just another job board. With them, it becomes a trusted infrastructure layer for the open-source development economy.

Gold Tier Target Justification

4 specialized AI agents in a production LangGraph pipeline
3 RAG use cases with hybrid retrieval + reranking
Constitutional AI-inspired self-reflection for fairness
Full OWASP LLM Top 10 coverage with specific mitigations
Langfuse observability + 5 quality metrics + LLM-as-a-Judge
Arabic language + RTL UI + voice input (bilingual platform)
100+ case evaluation dataset with ground-truth labels
MCP server roadmap for IDE integration (emerging technology)